Using Large Language Models (LLMs) in Autonomous Vehicles (AVs)

Caleb Bergen
CSE 198 (AIEA Lab)
Department of Computer Science and Engineering
University of California, Santa Cruz

I. Introduction

Autonomous vehicles offer several clear advantages, such as improved safety, reduced congestion and vehicle emissions, and greater accessibility [1]. Companies like Zoox and Waymo have been developing autonomous vehicle technology for years now, and they have been expanding access to these vehicles over time. While these technologies have improved significantly, they are still not perfect, and there have been serious accidents resulting from decisions made by these vehicles. One major example occurred in San Francisco in October of 2023. A pedestrian crossing the street was struck by a human driven car, and the collision sent the pedestrian into the path of a Cruise autonomous vehicle. While the Cruise vehicle did initially stop, it then tried to pull over while the pedestrian was in front of it and ended up dragging the pedestrian along the road for another 20 feet before ultimately stopping. The pedestrian was injured even more severely as a result of the actions taken by the Cruise vehicle. As a result of this accident, Cruise is no longer permitted to have autonomous vehicles driving in the state of California [2]. This is a more severe example of a mistake made by an autonomous vehicle, but it is important that these technologies be further developed to prevent similar accidents.

The accidents that do occur generally happen in situations that were not originally included in the training data because they were not anticipated. These are difficult to prepare autonomous vehicles for due to how they learn. Autonomous vehicle systems learn by being exposed to a variety of scenarios and identifying what constitutes a good action in those situations. They can then apply what has worked in the past to the current situation, but this is not guaranteed to be effective. In many scenarios in which autonomous vehicles cause or worsen traffic accidents, it is reasonable to say that a human would have responded in a safer manner due to our more advanced reasoning capabilities. This is where large language models (LLMs) like ChatGPT can be useful. Since they have been trained on large amounts of human generated information, they retain some of the human like reasoning present in that data. As a result, researchers have begun to examine where LLMs can be used in autonomous vehicle scenarios and how effective they are.

II. Background

A. LLM Decision Making Engines

One of the main ways LLMs are being applied to autonomous vehicle systems is as the primary decision making agent. Each of the following approaches follows a process similar to the one described below.

The LLM is fed information about the road it is on, and it has to decide what the safest action is. The environment the LLMs are exposed to is currently simulated. The technology is not currently advanced enough for real world implementation, so a simulation must be used to create the road it drives on. Once the LLM decides what to do, that action is taken in the environment. This has the car it controls, as well as any other cars on the road, follow their chosen actions for a set period of time. This is usually about a second, and at the end of that second, the LLM is called to choose an action once again. Once the LLM collides or a set amount of time elapses, the environment resets, and this sequence of actions is called an episode. Many episodes are run in sequence to obtain a larger data sample for examining the LLMs performance.

1) DiLu: Wen et al. [3] developed a framework, DiLu, capable of making driving decisions through an LLM centered reasoning loop. The framework consists of four modules: environment, reasoning, reflection, and memory, which combine to enable self reflection. The four key modules of the framework are pictured in Fig. 1.

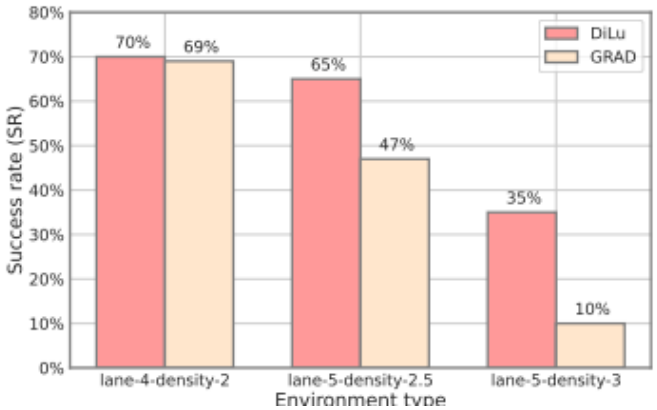

Fig. 1. Breakdown of the 4 modules in the DiLu framework. [3]

The environment is a simulated highway that tracks the position of the LLM controlled vehicle and controls other vehicles on the road, simulating normal traffic. A prompt is created for the LLM in the reasoning module using the current description of the environment and relevant experiences from the memory module. When the LLM responds to the prompt, it specifies the action to take, and that action is then executed in the environment. This cycle repeats itself until a set amount of time passes, or the LLM controlled vehicle collides with another vehicle [3].

The memory module is a key component of the LLMs function, as it randomly stores a few actions taken during the episode for later recall as context for decision making. Actions taken in episodes where no collision happens are kept in their original state, as this behavior was good and should be reinforced. Actions taken during episodes in which the LLM controlled vehicle collides are not immediately stored in the memory module. Instead, the actions taken and the reasoning output from the LLM are returned to it, and it is tasked with identifying where the incorrect decisions originated and finding a better course of action. The revised decision and reasoning are then stored in the memory module to ensure the LLM can learn from its own mistakes and limit their repetition. This type of memory is meant to emulate how humans remember experiences and use them to determine what to do or not do in any scenario [3]. This also enables

the LLM to improve its decision making over time, as it accumulates more driving experience.

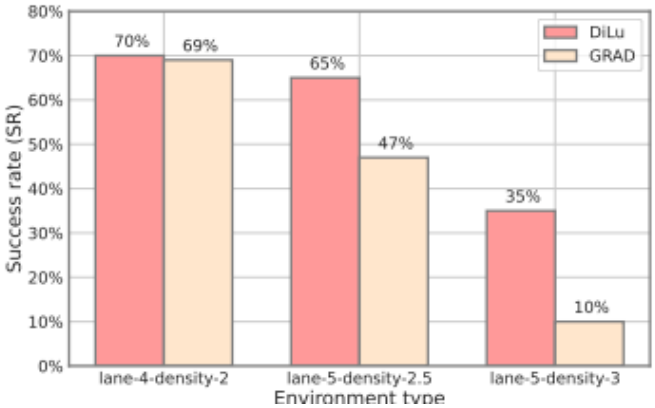

Fig. 2. The number after lane represents how many lanes are on the road, and the number after density represents how dense the traffic is. The traffic density value doesn't directly correspond to a real world value, it instead signifies the relative amount of traffic that will be created, where the higher the number is the more traffic there is. [3]

Wen et al. tested the DiLu framework by comparing it against the reinforcement learning algorithm GRAD, and the results are shown in Fig. 2. GRAD was trained over 600,000 timesteps in the lane-4-density-2 environment, while DiLu was trained over 40 episodes. The success rate in the graph represents the percentage of the time that the vehicle is not involved in a collision during the 30 second episode. Both DiLu and GRAD perform about the same in the first environment, but GRAD performs much worse in the other two environments. These results make sense because reinforcement algorithms are good at learning to perform well in whatever environment they are placed in, but can struggle to apply what they have learned in one environment to another. LLMs, on the other hand, have a higher baseline knowledge, so they perform better in unfamiliar environments. They also take less time to train, which is why they were given significantly fewer training actions.

Methods	MIN	Q1	Median	Q3	MAX
baseline	2.0	3.0	10.0	22.0	30.0
+success_memory	3.0	7.0	24.5	29.3	30.0
+correction_memory	4.0	7.75	21.5	30.0	30.0
+both_type_memory	**5.0**	**7.8**	**24.5**	**30.0**	**30.0**

Fig. 3. Number of seconds the DiLu agent made it through the 30 second episode based on what type of memory it used. [3]

Additionally, Wen et al. reported results for using memory banks versus not using them in Fig. 3. The success memory includes all actions taken by the LLM that were correct originally, and the correction memory includes the actions that needed to be corrected. The last type of memory listed in Fig. 3 combines both the success and correction memory into one memory bank. Without any memory, the agent can make it one third of the way through an episode on average. However, performance improves significantly when any type of memory is used. The success memory and combined memory have the same median score, and the correction memory and combined memory have the same Q3 and max performance. As expected though, the combined memory performs best among the others since it provides the most information to the LLM. In terms of decision making capabilities, this shows that the agent improves as it is given more information.

2) Agent-Driver: Mao et al. created a framework, called Agent-Driver, with a vast tool library that the LLM can call on to acquire additional context about the environment. Those functions are not automatically called. Instead, the LLM chooses which to call, making the program more efficient and reducing the time it takes to run. Mao et al. broke the driving process down into three stages which are perception, prediction, and planning [4].

Perception involves the agent detecting other vehicles and objects around it to avoid collisions. Some of the perception functions in the tool library involve finding the closest vehicle in front of the agent and all vehicles within a certain radius [4].

Prediction involves the agent thinking about where vehicles and objects on the road will be in the future. While driving, it is important to consider where cars will end up, and the prediction is meant to emulate that process. Some of the prediction functions include predicting the future speeds of other vehicles and the likelihood that a specific location will be occupied at a given time [4].

Lastly, there is planning, which involves the agent making a plan for what actions it will execute over a certain period of time. Certain aspects of driving, such as passing another vehicle, require multi step planning to execute effectively. Some functions the agent uses for planning involve finding open sections of the road and checking how close the agent is to the ends of the lane it is in or to the curb [4].

Beyond the three stage process for choosing actions, Agent-Driver implements several additional features to improve robustness and reasoning. Similar to DiLu, Agent-Driver implements a self reflection check on its planned actions. Once the LLM has planned the actions it will take, predictions are made at each point in time to determine whether it will collide with another vehicle or come too close to one. If that is the case, the LLM is tasked with considering an alternative action to ensure a safer driving path [4].

Also like DiLu, Agent-Driver has a memory module that can leverage knowledge from previous experiences. Part of the memory module stores past experiences, which enable the agent to repeat successful actions. There is also an initial commonsense memory bank that provides basic driving guidelines and instructions for the LLM to follow. These include traffic regulations and knowledge of risky driving behaviors, and what is included is fully customizable [4].

Mao et al. compare the performance of Agent-Driver against several other agents in Fig. 4. All the other agents are previ-

ously created LLM driven agents made by other researchers.

	Method	L2 (m) ↓				Collision (%) ↓			
		1s	2s	3s	Avg.	1s	2s	3s	Avg.
ST-P3 metrics	ST-P3 (Hu et al., 2022)	1.33	2.11	2.90	2.11	0.23	0.62	1.27	0.71
	VAD (Jiang et al., 2023)	0.17	0.34	**0.60**	0.37	0.07	0.10	0.24	0.14
	GPT-Driver (Mao et al., 2023a)	0.20	0.40	0.70	0.44	0.04	0.12	0.36	0.17
	Agent-Driver (ours)	**0.16**	**0.34**	0.61	**0.37**	**0.02**	**0.07**	**0.18**	**0.09**
UniAD metrics	NMP (Zeng et al., 2019)	-	-	2.31	-	-	-	1.92	-
	SA-NMP (Zeng et al., 2019)	-	-	2.05	-	-	-	1.59	-
	FF (Hu et al., 2021)	0.55	1.20	2.54	1.43	0.06	0.17	1.07	0.43
	EO (Khurana et al., 2022)	0.67	1.36	2.78	1.60	0.04	0.09	0.88	0.33
	UniAD (Hu et al., 2023)	0.48	0.96	1.65	1.03	0.05	0.17	0.71	0.31
	GPT-Driver (Mao et al., 2023a)	0.27	0.74	1.52	0.84	0.07	0.15	1.10	0.44
	Agent-Driver (ours)	**0.22**	**0.65**	**1.34**	**0.74**	**0.02**	**0.13**	**0.48**	**0.21**

Fig. 4. Comparison between Agent-Driver and other LLM based driving systems. [4]

A loss function is used to determine how well LLM agents perform and to assign a numerical value to their performance. L2 loss or mean squared error (MSE) is used here. The mean squared error of each agent is compared at 1, 2, and 3 seconds. At almost every point, Agent-Driver performs better than all other agents, and its average MSE is better than all others. The other part of the figure examines the collision percentage of the agents after 1, 2, and 3 seconds. Once again, Agent-Driver performs better than the other listed agents. This shows an advancement in the reasoning capabilities of the Agent-Drivers system, either through a smarter LLM or a better designed framework.

3) KoMA: Jiang et al. created a framework called KoMA (Knowledge-driven Multi-agent Framework for Autonomous Driving with Large Language Models). KoMA is similar to DiLu and Agent-Driver because it supports a single driving agent and uses a memory system to refine LLM actions over time. On top of that, it supports a multi agent framework with multiple LLM controlled vehicles on the simulated road [5].

KoMA functions similarly to DiLu and Agent-Driver, where all information about the environment is fed to the LLM, along with additional context in the form of background prompts and memory from previous experiences. Each LLM agent functions separately, and they are unaware that the others are any different from a vehicle controlled by the environment while they are driving. At the end of each episode, experiences from both agents are added to a shared memory bank that both can use [5]. This allows each agent to learn from the successes and mistakes of the others, while also testing the behavior of each LLM agent. Each LLM reacts to the other's behavior as if it were any other vehicle on the road.

Another important addition to KoMA is a multi step reasoning process that all of the agents use. Instead of processing all the details and making a decision in one go, the LLM's reasoning is split into several steps. The LLM first clarifies the goals it has based on the current scenario. The main goals will often be to drive safely and efficiently, but they might also include yielding to pedestrians in a crosswalk or yielding to let other cars merge when passing an on ramp. Next, the LLM develops a plan for what it needs to do to work towards its goal. Once the LLM has formulated a plan of what it will do, it decides which action to take first to begin executing it [5]. The multi step process also gives the LLM more time to reflect on its decision, which can help it produce a more accurate response.

In Fig. 5, Jiang et al. compare the performance between no memory, non shared memory, and shared memory. No memory performs the worst, which is consistent with the results from DiLu and Agent-Driver. Non shared memory performs better because it is essentially the same type of memory that DiLu and Agent-Driver use, since each agent stores its own memories. The shared memory agents perform best, and the difference becomes clear after 40 episodes. This is likely because the shared memory bank fills faster than the non shared one, so the LLM will receive additional context and experience more quickly. This provides the LLM with a greater wealth of knowledge to draw on when making decisions.

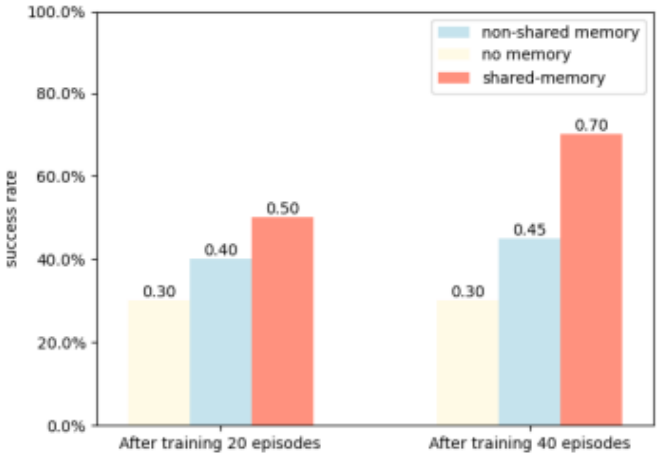

Fig. 5. Comparison between LLM agents using each listed type of memory. [5]

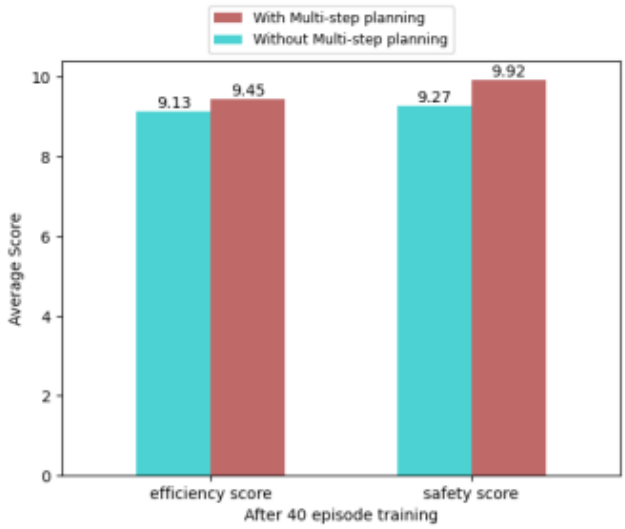

Fig. 6. Comparison between LLM agent performance with and without multi step planning. [5]

Jiang et al. also provide a comparison between using multistep planning and not using it in Fig. 6. They use their own efficiency and safety score to compare the performance across those two metrics. The maximum score for each metric is 10, and whether agents use multistep planning or not, they still get similarly high scores. That said, using multi step planning does result in slightly higher scores in both metrics. Jiang et al. attribute this to multistep planning, which allows for more thorough processing of information [5].

III. Methods

A. Program Structure

The control loop my program follows is depicted in Fig. 7. Everything starts with the simulated environment. It keeps track of and displays the ego vehicle as well all other simulated vehicles on the road. The ego vehicle is just another name for the LLM controlled vehicle. At each timestep the environment outputs data about the positions and speeds of all of the vehicles on the road. There is a small processing step applied to the data which is then combined with static system prompts to be given to the LLM. Once the LLM received all of the prompts and data, it makes a decision on what action should be taken, which is given to the environment. Once the environment receives the action from the LLM it updates all vehicles positions, and starts the process over again by outputting the updated positions and speeds for all of the vehicles. This process is repeated for a total of 40 timesteps, or until the ego vehicle is involved in a collision, whichever happens first.

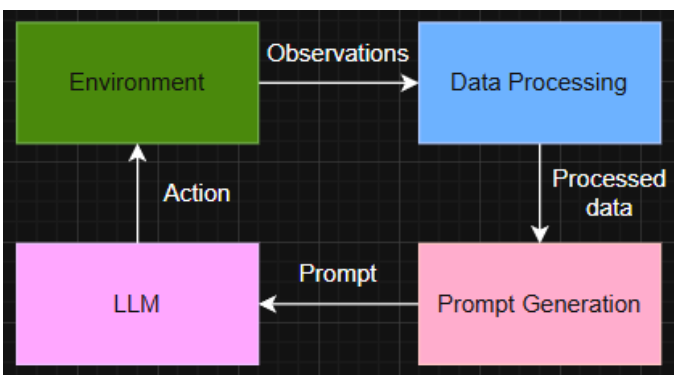

Fig. 7. Control loop for my program

B. Environment

The environment I used was the HighwayEnv environment made by the Farama foundation. The environment by default is a four-lane highway that has one user-controlled vehicle, and many other vehicles that are controlled by the environment. At each timestep the environment outputs a list containing the speeds and positions of all vehicles on the road. The first vehicle in the list is the ego vehicle. The four next closest vehicles to the ego vehicle are also included in the list, labeled Vehicle 1 through Vehicle 4 where the first vehicle is the closest to the ego vehicle.

C. Data Processing

Instead of passing the list from the environment directly to the LLM, I put all of the data into a data class before passing it on. The purpose of using the data class was to make it clearer what each element of the list corresponded to. Without the data class it was unclear when examining the code what each element of the list of data corresponding to a vehicle represented. Including the data class allows me to specify the name of each attribute when storing them to a new list. Once the list from the environment has been fully converted to the data class it is ready to be included in the prompt.

D. Prompt Generation

The first part of the prompt given to the LLM is a base prompt that specifies what the LLM is tasked with doing and how it should respond to everything else it will receive. The base prompt given to the LLM is as follows: ”You are an intelligent driving assistant whose goal is to drive safely and efficiently. You are directing the ego vehicle in this simulation, and your job is to select the best action given a list of possible actions and the state space at a specific time step. Explain your reasoning thoroughly for each candidate action before selecting the most optimal action.” After the LLM is prompted with the base prompt, it then receives all of the data about the vehicles in the environment. Lastly, it is given a list of which actions it can take. This list reflects what lane it is in, so it can’t change lanes to the left if it is already in the left most lane, and it tells the LLM what numbers it needs to output that correspond to the action it wants to take.

E. LLM

Originally I used GPT-5-nano as it was the fastest GPT-5 model at the time. I wanted to ensure that the model I chose had the capabilities to reason well about what actions to take. After using this model, I got good results in its reasoning capabilities; however, it would take around 20 seconds on average to choose a decision for a single timestep. This is not very practical as a lot can change on a road in 20 seconds, so decisions would need to be made much faster than this. I realized that William used GPT-4o in his research for its speed, so I gave it a try to see how much that would change things. Switching to GPT-4o reduced response times from 20 seconds on average to about 5 to 10 seconds, which is a noticeable decrease but still needs to be faster for this to be practical.

One aspect I had problems with at first was getting a consistent numerical output of what action should be taken. Initially I prompted GPT to make the first character of its response be the action that should be taken. This proved to work most of the time, but sometimes the response wouldn’t follow the requested pattern which would result in the program crashing. After some research I found that you can structure how the response is given from GPT, and this was able to fix the problem. I had the response that contained the reasoning for the choice of action taken given as a separate output from the numerical action to take. This ensured that the correct

action was able to be identified separately from the response, and ensured the proper action would be taken.

IV. Results

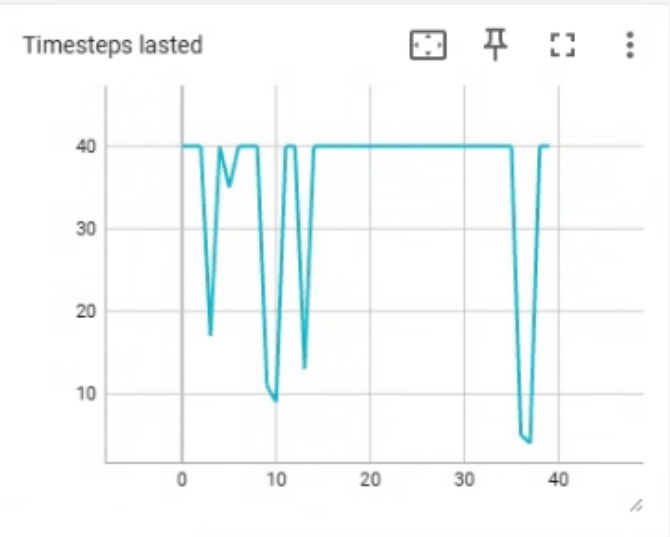

Fig. 8. Number of timesteps the agent made it through each episode.

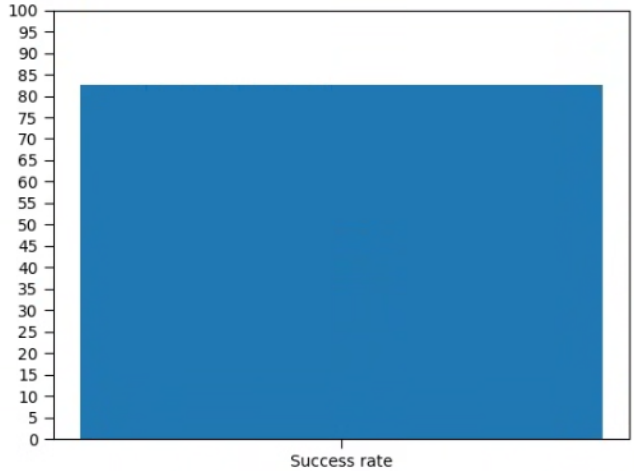

Fig. 9. Percentage of episodes that did not end with a collision.

To test my framework I ran it for 40 episodes to see how ChatGPT would perform over an extended period of time. Fig. 8 shows the number of timesteps the agent navigated before collision. The episode automatically concludes at 40 timesteps, so in those episodes the agent did not crash. In 33 out of the 40 total episodes the agent was able to navigate all 40 timesteps without collisions. Fig. 9 shows the success rate of the agent in a percentage, where the success rate is the percent of the time the agent does not collide with another vehicle during an episode. The success rate the agent achieved over 40 episodes was 82.5%.

V. Discussion and Conclusion

I took inspiration from the Dilu framework proposed by Mao et al. when designing mine. My program uses a control loop similar to DiLu, and the primary difference between my implementation and DiLu is that my program has not yet implemented any type of memory for the LLM. Mao et al. present results showing that including a memory module significantly improves agent performance. I expected my results to be slightly worse than DiLu because my agent doesn't have memory capabilities. The opposite ending up being the case, as DiLu had a 70% success rate over 40 episodes while my agent had an 82.5% success rate. Both DiLu and my agent used the same number of lanes on the road and had the same traffic density, so there was no difference there.

There are a couple reasons that I think could have caused my agent to perform better than DiLu. The first reason could be the ChatGPT model that was used in each of the frameworks. DiLu used GPT-3.5 for reasoning and GPT-4 for reflection, while my agent uses GPT-4o. Per OpenAIs website, GPT-4o has the highest intelligence out of the 3 models. GPT-4o is a more recently released model than the other two which is likely why it has better reasoning capabilities. GPT-4o having higher intelligence means it likely is able to reason better in driving scenarios and take better actions on average.

Another cause could be the driving behavior of my agent. In most of the episodes I have examined, my agent tends to drive very passively to avoid a collision. It will often stay behind whatever vehicles are in front of it, and make no attempts to pass them. This isn't inherently a bad behavior, since it isn't necessary to pass all other vehicles. That said, the agent should still have the behavior to pass vehicles when necessary, which doesn't seem to happen very often. This sort of passive driving likely contributes to a lower rate of collisions, and also reduces the speed the agent drives at.

An additional aspect that improved the performance of my agent was using a data class to store the data for each of the vehicles. Initially I implemented the data class simply for readability purposes, since it made it clearer what each value corresponds to. However, after implementing it there was a noticeable increase in the accuracy of decisions taken by the LLM. Before I was using the data class there were often times where the LLM seemed to not realize there was another vehicle close to it, and it would either accelerate or change lanes into it. Previously, as part of the prompt the LLM was told what each value corresponded to before processing the entire list of pure numbers. Now with the data class, when the LLM is processing the data of the vehicles it reads the text specifying what each value corresponds to as it is reading through the list. I think the more consistent format of the data class, as well as the added context, both contribute to the improved performance of the agent.

Over this past quarter my agent has significantly improved, but there are still areas where I could improve within the framework. Future areas of improvement include fine tuning the driving behavior of the agent, and finding a way to increase

the speed at which the LLM processes information. Progress in both of these areas will move the program closer to being physically viable in the real world.

Like I mentioned previously, the current agent's behavior appears to be very passive. Not every scenario benefits from driving passively, and maneuvers such as merging and passing other vehicles involve driving in a less passive manner. These are normal driving maneuvers that sometimes need to be executed to drive in the most efficient manner. An important challenge will be finding the appropriate balance between maintaining safety through passive driving and allowing the agent to confidently execute necessary maneuvers.

The LLM will need to be able to process data from the environment and make a decision in fractions of a second to be viable as the main decision making element. Some ways I could speed up the runtime include finding faster LLM models and improving the efficiency of the program. I could also use a hybrid approach instead of using the LLM for every driving decision that needs to be made. A system like this would involve using traditional AV systems for most driving scenarios, with the LLM able to be used in a more advisory / evaluation role and as needed in high danger scenarios. This type of system combines the speed and reliability of traditional methods with the higher level reasoning capabilities of LLMs.

References

[1] "Benefits of Autonomous Vehicles (AVs)," autosinnovate.org. [Online]. Available: https://www.autosinnovate.org/initiatives/innovation/autonomous-vehicles/benefits-of-havs. [Accessed Mar 6, 2026]

[2] K. Truong, "Cruise hid video of woman being dragged along San Francisco street, DMV says, " sfstandard.com. [Online]. Available: https://sfstandard.com/2023/10/24/cruise-robotaxi-dmv-suspension-video/. [Accessed Mar 6, 2026]

[3] L. Wen, D. Fu, X. Li, X. Cai, T. Ma, P. Cai, M. Dou, B. Shi, L. He, Y. Qiao, "DiLu: A Knowledge-Driven Approach to Autonomous Driving with Large Language Models," September 28, 2023. [Online]. Available: https://arxiv.org/abs/2309.16292 [Accessed: Feb 15, 2026].

[4] J. Mao, J. Y, Y. Qian, M. Pavone, Y. Wang, "A Language Agent for Autonomous Driving," [Online]. Available: https://arxiv.org/abs/2311.10813. [Accessed: Feb 15, 2026]

[5] K. Jiang, X. Cai, Z. Cui, A. Li, Y. Ren, H. Yu, H. Yang, D. Fu, L. Wen, P. Cai, "KoMA: Knowledge-driven Multi-agent Framework for Autonomous Driving with Large Language Models," [Online]. Available: https://arxiv.org/abs/2407.14239 [Accessed: Feb 15, 2026]

[6] A. Guo, J. Hou, and F.-Y. Wang, "BladeRunner-AD: Adversarial Coevolution of Robust and Interpretable Driving Policies via LLM Agent Games," in Mechtronic and Embedded Systems and Applications, MESA, IEEE/ASME International Conference on, IEEE, 2025, pp. 122–127. doi: 10.1109/MESA68091.2025.11278881. [Accessed: Feb 15, 2026]

[7] Y. Tang, L. Moffat, W. Guo, C. May-Chahal, J. Deville, and A. Tsourdos, "Encoding Social & Ethical Values in Autonomous Navigation: Philosophies Behind an Interactive Online Demonstration," in Proceedings of the Second International Symposium on Trustworthy Autonomous Systems, New York, NY, USA: ACM, 2024, pp. 1–9. doi: 10.1145/3686038.3686044. [Accessed: Feb 15, 2026]

[8] R. Alake, "Loss Functions in Machine Learning Explained, " datacamp.com. [Online]. Available: https://www.datacamp.com/tutorial/loss-function-in-machine-learning. [Accessed Mar 6, 2026]

[9] C. Stryker, "What are large language models (LLMs)?, " ibm.com. [Online]. Available: https://www.ibm.com/think/topics/large-language-models. [Accessed Mar 6, 2026]